
Análise técnica do capítulo 2 - Build a Large Language Model

Inteligência Artificial e Sistemas Inteligentes

Ana Clara V. P. de Oliveira (412887) e Jasmine Masson Alves (427466)

1 Introdução

Antes que o modelo consiga aprender relações entre palavras, compreender contexto ou prever o próximo token, o texto precisa passar por uma sequência de transformações que o convertem de linguagem humana em dados numéricos que possam ser processados matematicamente.

No Capítulo 2 do livro *Build a Large Language Model (From Scratch)*, de Sebastian Raschka, é apresentado justamente esse processo de preparação dos dados. O capítulo aborda desde a tokenização, passando pela criação do vocabulário e conversão dos tokens em Token IDs, até a construção dos pares de entrada e alvo, a organização dos dados em batches e a transformação dos identificadores em embeddings. Também é introduzida a necessidade de representar a posição dos tokens, preparando os dados para serem processados pela arquitetura Transformer.

Nessa análise, o objetivo não é apenas reproduzir os códigos apresentados no livro, mas compreender a relação entre cada etapa e o motivo pelo qual ela é necessária. Essa visão é fundamental porque cada etapa resolve uma limitação da anterior. O texto bruto não pode ser utilizado diretamente pelas operações matemáticas da rede neural, os Token IDs identificam os tokens, mas não representam seu significado, os embeddings transformam esses identificadores em vetores aprendíveis e a informação posicional permite preservar a ordem dos elementos.

Dessa forma, o Capítulo 2 estabelece a base para compreender o funcionamento dos LLMs. O resultado desse processamento será utilizado nas etapas seguintes do projeto, especialmente no mecanismo de atenção, responsável por permitir que o modelo estabeleça relações entre diferentes tokens de uma mesma sequência.

2 Análise Técnica dos experimentos e resultados

Os experimentos realizados nesta Sprint tiveram como objetivo compreender, de forma prática, como um texto bruto é transformado em uma representação numérica adequada para ser processada por uma Large Language Model (LLM). Os procedimentos foram baseados nos conceitos apresentados no Capítulo 2 do livro *Build a Large Language Model (From Scratch)*, especialmente nas etapas de tokenização, conversão para Token IDs, criação das sequências de

entrada e alvo, organização dos dados em batches, transformação dos tokens em embeddings e incorporação da informação posicional.

Uma LLM não pode trabalhar diretamente com o texto bruto porque o texto é uma informação categórica e as operações realizadas pelas redes neurais dependem de valores numéricos. Dessa forma, é necessário transformar progressivamente o texto em uma representação que possa ser utilizada nas operações matemáticas da rede neural.

O pipeline estudado nesta Sprint pode ser representado de forma simplificada como:

Texto bruto $\rightarrow$ *Tokens* $\rightarrow$ *Token IDs* $\rightarrow$ *Sequências de entrada e alvo* $\rightarrow$ *Batches* $\rightarrow$ *Token Embeddings* $\rightarrow$ *Positional Embeddings* $\rightarrow$ *Input Embeddings*

Essa preparação é fundamental para que, na etapa seguinte, os dados possam ser processados pelos mecanismos do Transformer, especialmente pelo mecanismo de self-attention.

2.1 Tokenização e Token IDs

A primeira etapa analisada foi a tokenização. O objetivo da tokenização é dividir o texto em unidades menores chamadas tokens. Dependendo do tokenizador utilizado, um token pode representar uma palavra inteira, parte de uma palavra, um caractere ou um símbolo especial.

Nos experimentos foi utilizado o tokenizador BPE (*Byte Pair Encoding*) disponibilizado pela biblioteca `tiktoken`, utilizando a codificação GPT-2 tokenizer = `tiktoken.get_encoding("gpt2")`

O BPE possui uma característica importante, uma palavra que não esteja presente como uma unidade completa no vocabulário pode ser dividida em subpalavras ou unidades menores. Dessa maneira, o tokenizador consegue representar palavras novas sem depender necessariamente de um token especial de palavra desconhecida, como o `<|unk|>`.

O experimento utilizou quatro frases diferentes e comparou a quantidade de caracteres com a quantidade de tokens produzidos. Também foi realizada a tokenização completa do arquivo `the-verdict.txt`.

Os resultados demonstram que a quantidade de tokens não é necessariamente igual à quantidade de caracteres ou palavras. Uma mesma palavra pode ser representada por um único token ou por vários tokens, dependendo da forma como o algoritmo BPE realizou as subdivisões e combinações durante a construção do vocabulário.

Token $\rightarrow$ *unidade textual utilizada pelo modelo*

Após a tokenização, cada token precisa ser convertido em um número inteiro. Esse número é chamado de Token ID.

Token $\rightarrow$ *Token ID*

O Token ID funciona como um identificador ou índice utilizado para localizar o token correspondente no vocabulário e, posteriormente, localizar seu vetor na matriz de embeddings.

Portanto, token e Token ID não são a mesma coisa. O token representa a unidade de texto, enquanto o Token ID é sua representação numérica. O vocabulário estabelece o mapeamento entre essas duas representações.

$$\text{Token} \leftrightarrow \text{Token ID}$$

O vocabulário inverso realiza o processo contrário, permitindo transformar Token IDs novamente em tokens e, posteriormente, reconstruir o texto.

Figura 1 - Tokenização

EXPERIMENTO 1 - TOKENIZAÇÃO			
Texto	Caracteres	Tokens	

Hello, how are you?	19	6	
The model learns from data.	27	6	
Machine learning uses numerical representations.	48	6	
This is a longer sentence used to observe how the number of tokens changes.	75	15	
The Verdict:			
Caracteres: 20479			
Tokens: 5145			

Fonte: Autoria própria.

2.2 Preparação das Sequências de Entrada e Alvo

Depois que o texto é transformado em Token IDs, é necessário organizar esses IDs em exemplos que possam ser utilizados durante o treinamento da LLM. Para isso, é utilizada uma janela deslizante sobre a sequência de tokens. A partir de um bloco de entrada, é criado um bloco alvo deslocado uma posição para frente.

Por exemplo, considerando a sequência:

$$[5, 6, 12, 20]$$

uma possível entrada pode ser:

$$[5, 6, 12]$$

enquanto o alvo correspondente será:

$$[6, 12, 20]$$

Dessa forma, cada posição do alvo corresponde ao próximo token que o modelo deve aprender a prever.

Essa organização está diretamente relacionada ao funcionamento de modelos autoregressivos, como os modelos GPT. Durante o treinamento, o modelo recebe os tokens anteriores e aprende a prever o próximo token.

O código utilizado nesta Sprint implementa essa preparação por meio da classe `GPTDatasetV1`, presente no arquivo `Entradas_Alvos.py`.

O parâmetro `max_length` determina o tamanho da sequência de contexto, enquanto o parâmetro `stride` determina quanto a janela avança para produzir a próxima amostra.

Quando:

`stride < max_length`

existe sobreposição entre as janelas.

Quando:

`stride = max_length`

não existe sobreposição entre as janelas.

Nos experimentos desta Sprint foi utilizado a segunda opção. Portanto, as janelas utilizadas nos testes não apresentam sobreposição.

2.3 Relação entre Tamanho do Contexto e Quantidade de Amostras

O segundo experimento avaliou diferentes tamanhos de contexto:

4, 8, 16, 32 e 64 tokens

Para cada valor de `max_length`, o mesmo texto foi utilizado para construir o dataset.

Os resultados permitem observar uma relação inversa entre o tamanho do contexto e a quantidade de amostras produzidas a partir de um texto de tamanho fixo. Quando o contexto é menor, cada janela utiliza uma menor quantidade de tokens, permitindo construir mais sequências.

À medida que o tamanho do contexto aumenta, cada amostra utiliza mais tokens. Consequentemente, uma quantidade menor de janelas pode ser extraída do mesmo texto.

Como o experimento utilizou `stride` igual ao tamanho do contexto, não houve sobreposição entre as janelas. Assim, o aumento do contexto resultou diretamente em uma redução na quantidade de amostras obtidas.

Esse resultado evidencia uma relação importante entre o tamanho do contexto e a quantidade de exemplos disponíveis para treinamento.

Um contexto maior permite que o modelo considere uma quantidade maior de tokens simultaneamente, possibilitando que relações mais distantes dentro da sequência sejam analisadas. Entretanto, quando o texto de origem possui tamanho fixo, um contexto maior reduz a quantidade de janelas independentes que podem ser obtidas.

Figura 2 - Contexto

EXPERIMENTO 2 - TAMANHO DO CONTEXTO		
Contexto	Tokens por sequência	Amostras
4	4	1286
8	8	643
16	16	321
32	32	160
64	64	80

Fonte: Autoria própria.

2.4 Organização dos Dados em Batches

Após a criação das sequências de entrada e alvo, os dados precisam ser organizados para serem fornecidos ao modelo durante o treinamento.

Essa função é realizada pelo `DataLoader`. Em vez de fornecer uma sequência individual ao modelo a cada operação, o `DataLoader` agrupa várias sequências em um batch.

Uma sequência individual pode ser representada como:

$[contexto]$

Enquanto um batch pode ser representado como:

$[batch, contexto]$

No experimento foi utilizado um contexto de tamanho 8 e foram avaliados os seguintes valores de `batch_size`:

$1, 2, 4, 8 \text{ e } 16$

Por exemplo, quando `batch size = 4` e `contexto = 8` a estrutura dos tensores de entrada e alvo apresenta o formato:

$[4, 8]$

Isso significa que quatro sequências, cada uma contendo oito Token IDs, são processadas simultaneamente.

O `DataLoader` também permite controlar outros aspectos da organização dos dados, como `shuffle`, `drop_last` e `num_workers`.

No experimento, foi utilizado `shuffle=False` e `droplast=True`

O parâmetro `drop_last=True` faz com que um último batch incompleto seja descartado quando o número total de amostras não é divisível pelo tamanho do batch.

O tamanho do batch também influencia o treinamento. Batches menores exigem menos memória, porém produzem atualizações mais frequentes e podem apresentar maior variabilidade. Batches maiores utilizam mais memória, mas permitem processar uma quantidade maior de exemplos em cada atualização.

Dessa forma, o `DataLoader` atua como uma etapa intermediária fundamental entre o dataset preparado e o modelo que realizará o treinamento.

Figura 3 - Lotes

EXPERIMENTO 3 - TAMANHO DO BATCH			
Batch size	Shape inputs	Shape targets	Batches
1	(1, 8)	(1, 8)	643
2	(2, 8)	(2, 8)	321
4	(4, 8)	(4, 8)	160
8	(8, 8)	(8, 8)	80
16	(16, 8)	(16, 8)	40

Fonte: Autoria própria.

2.5 Transformação dos Token IDs em Embeddings

Após a organização dos Token IDs, o próximo passo da preparação é transformá-los em vetores contínuos.

Essa transformação é necessária porque os Token IDs são valores numéricos discretos que funcionam como índices, mas não representam diretamente relações semânticas entre os tokens.

Por exemplo, o fato de um token possuir ID 100 e outro possuir ID 101 não significa que esses tokens sejam semanticamente mais próximos do que um token de ID 100 e outro de ID 500.

Para resolver esse problema, utiliza-se uma camada de embedding.

No código foi utilizada a camada `torch.nn.Embedding`

Essa camada pode ser entendida como uma operação de busca. Cada Token ID é utilizado para localizar uma linha específica na matriz de pesos da camada de embedding. Essa linha corresponde ao vetor que representa aquele token.

Assim:

$$\text{Token ID} \rightarrow \text{vetor de embedding}$$

A camada de embedding realiza, portanto, uma transformação de uma representação discreta em uma representação vetorial contínua.

Essa abordagem pode ser relacionada à ideia de codificação one-hot. Em vez de criar

explicitamente um vetor one-hot enorme e realizar uma multiplicação matricial, a camada de embedding realiza diretamente a busca da linha correspondente na matriz de pesos.

No experimento foi utilizado um vocabulário com:

50.257 tokens

e foram avaliadas quatro dimensões diferentes de embedding:

64, 128, 256 e 512

A estrutura inicial de Token IDs possuía o formato:

[8, 4]

Após a transformação em embeddings, o formato passa a ser:

[8, 4, dimensão_do_embedding]

Assim, para uma dimensão de 256:

[8, 4, 256]

Isso demonstra que a dimensão do embedding altera diretamente a estrutura dos dados que será utilizada pelas camadas seguintes.

2.5.1 Impacto da Dimensão do Embedding

A dimensão do embedding também altera diretamente a quantidade de parâmetros da camada.

A quantidade de parâmetros pode ser calculada por:

$$\text{Parâmetros} = \text{tamanho do vocabulário} \times \text{dimensão do embedding}$$

Considerando um vocabulário de 50.257 tokens:

$$50.257 \times 64 = 3.216.448$$

$$50.257 \times 128 = 6.432.896$$

$$50.257 \times 256 = 12.865.792$$

$$50.257 \times 512 = 25.731.584$$

Portanto, aumentar a dimensão do embedding aumenta tanto o tamanho dos vetores utilizados pelo modelo quanto a quantidade de parâmetros que precisam ser armazenados e treinados.

Dimensões maiores podem permitir que os embeddings representem mais nuances e relações, mas também aumentam o custo computacional e o consumo de memória.

Figura 4 - Embeddings

EXPERIMENTO 4 - DIMENSÃO DOS EMBEDDINGS		
Dimensão	Shape	Parâmetros
64	(8, 4, 64)	3216448
128	(8, 4, 128)	6432896
256	(8, 4, 256)	12865792
512	(8, 4, 512)	25731584

Fonte: Autoria própria.

2.6 Positional Embeddings

Os embeddings dos tokens fornecem informações sobre quais tokens estão presentes na sequência. Entretanto, existe uma limitação: a camada de embedding retorna o mesmo vetor para um determinado Token ID, independentemente da posição em que ele aparece.

Por exemplo, se determinado token possuir o ID 5, a camada de embedding retornará o mesmo vetor tanto quando o token estiver na primeira posição quanto quando estiver na última posição da sequência.

Isso representa um problema porque a ordem dos tokens possui importância fundamental na linguagem.

Considere as sequências:

O gato comeu

e:

Comeu o gato

Mesmo que os mesmos elementos estejam presentes, a ordem produz uma estrutura linguística diferente.

O mecanismo de self-attention consegue analisar as relações entre os tokens, mas a informação de posição precisa ser incorporada à representação para que o modelo possa diferenciar a localização de cada token.

Por isso, são utilizados os Positional Embeddings.

No experimento foram criadas duas posições diferentes para o mesmo Token ID:

$$\text{Token ID} = 100$$

nas posições:

$$0 \text{ e } 3$$

A representação final foi obtida pela soma:

$$\text{Input Embedding} = \text{Token Embedding} + \text{Positional Embedding}$$

Dessa forma, o token embedding fornece a informação relacionada ao token, enquanto o positional embedding fornece a informação relacionada à posição.

Pode-se interpretar essa combinação da seguinte forma.

Token Embedding = “o que é o token”

Positional Embedding = “onde está o token”

Input Embedding = “o que é + onde está”

O experimento confirmou essa característica. Mesmo utilizando o mesmo Token ID, os embeddings das posições 0 e 3 foram diferentes. Como consequência, as representações finais também foram diferentes.

2.6.1 Posições Absolutas e Relativas

Existem diferentes formas de representar a posição dos tokens.

Nos Positional Embeddings absolutos, cada posição possui uma representação própria. Dessa forma, o modelo recebe informações sobre a posição exata do token dentro do contexto.

Já os métodos relativos procuram representar principalmente a relação entre as posições. Em vez de indicar apenas a posição absoluta, podem representar relações como a distância entre dois tokens.

Ambas as abordagens possuem o objetivo de fornecer ao modelo informações relacionadas à ordem dos tokens.

O experimento desta Sprint utiliza explicitamente embeddings para as posições dentro do contexto, permitindo demonstrar a influência da posição na representação final.

Figura 5 - Posição Embeddings

```

EXPERIMENTO 5 - POSITIONAL EMBEDDINGS
Mesmo Token ID utilizado: 100
Posições comparadas: 0 e 3
Embeddings das posições são diferentes: True
Representações finais são diferentes: True

```

Fonte: Autoria própria.

2.7 Integração das Etapas da Pipeline

A análise dos experimentos demonstra que as etapas realizadas nesta Sprint não são independentes. Cada etapa prepara os dados necessários para a etapa seguinte.

Inicialmente, o texto bruto é dividido em tokens utilizando o tokenizador BPE. Em seguida, cada token é associado a um Token ID por meio do vocabulário.

Os Token IDs são então organizados em sequências de entrada e alvo por meio de uma janela deslizante. O alvo corresponde à sequência deslocada uma posição para frente, permitindo que o modelo seja treinado para prever o próximo token.

Posteriormente, o **DataLoader** organiza essas sequências em batches. Os Token IDs presentes nos batches são então convertidos em vetores contínuos por meio da camada de embedding.

Finalmente, embeddings posicionais são adicionados aos embeddings dos tokens. O resultado é uma representação denominada *input embeddings*, que contém informações sobre os tokens e sobre suas respectivas posições.

O pipeline completo pode ser representado por:

Texto bruto → **BPE** → **Tokens** → **Token IDs** → **Entrada/Alvo** → **DataLoader** →
Token Embeddings → **Positional Embeddings** → **Input Embeddings**

Essa representação é o resultado final da preparação dos dados realizada nesta Sprint.

2.8 Relação com o Mecanismo de Atenção da Próxima Sprint

As informações produzidas nesta Sprint serão utilizadas diretamente nas próximas etapas da arquitetura da LLM.

A principal informação produzida é o conjunto de *input embeddings*. Esses vetores são obtidos pela combinação dos embeddings dos tokens com os embeddings posicionais:

$$Input\ Embeddings = Token\ Embeddings + Positional\ Embeddings$$

Esses vetores representam, simultaneamente, quais tokens estão presentes na sequência e em quais posições eles estão localizados.

Na próxima Sprint, essas representações serão fornecidas às camadas do Transformer. O mecanismo de self-attention utilizará esses vetores para calcular relações entre os diferentes tokens presentes no contexto.

Assim, o trabalho realizado nesta Sprint constitui uma preparação necessária para o mecanismo de atenção. O self-attention não recebe diretamente o texto bruto nem os Token IDs como sua representação principal. Ele trabalha sobre as representações vetoriais produzidas pelas etapas anteriores.

Portanto, a relação entre as duas Sprints pode ser resumida como:

Sprint 2: preparação e representação dos dados

⇓

Input Embeddings

⇓

Sprint 3: Self-Attention e processamento pelo Transformer

3 Conclusão

Os experimentos realizados nesta Sprint permitiram compreender e verificar na prática as principais etapas necessárias para transformar um texto bruto em uma representação adequada para uma Large Language Model.

Inicialmente, foi possível verificar que uma LLM não pode trabalhar diretamente com texto bruto, pois as redes neurais realizam operações matemáticas sobre valores numéricos. A tokenização BPE mostrou como um texto pode ser dividido em unidades menores e convertido em Token IDs.

Também foi possível compreender a diferença entre tokens e Token IDs. Enquanto o token corresponde a uma unidade textual, o Token ID funciona como um identificador numérico utilizado para localizar essa unidade no vocabulário e, posteriormente, seu vetor correspondente na camada de embedding.

A preparação das sequências demonstrou como os Token IDs são organizados em pares de entrada e alvo. O deslocamento do alvo em uma posição permite criar os exemplos necessários para o treinamento autoregressivo, no qual a LLM aprende a prever o próximo token.

O experimento com diferentes tamanhos de contexto demonstrou que o aumento do contexto altera a quantidade de amostras que podem ser extraídas de um texto de tamanho fixo. Como nos experimentos o **stride** foi definido igual ao tamanho do contexto, não houve sobreposição entre as janelas.

A análise dos batches demonstrou a função do **DataLoader** na organização dos dados. O recurso permite agrupar várias sequências em um único batch, facilitando o fornecimento dos dados ao modelo durante o treinamento.

Os experimentos de embeddings demonstraram que os Token IDs não devem ser interpretados como representações semânticas. Eles são identificadores discretos utilizados para localizar vetores na matriz de embeddings. A camada de embedding realiza essa transformação, produzindo vetores contínuos que podem ser processados pelas redes neurais.

Também foi verificado que a dimensão do embedding possui impacto direto na estrutura dos dados e na quantidade de parâmetros. Para o vocabulário de 50.257 tokens utilizado no

experimento, aumentar a dimensão de 64 para 512 resultou em um aumento significativo na quantidade de parâmetros da camada.

Os experimentos com Positional Embeddings demonstraram a necessidade de representar a posição dos tokens. O mesmo Token ID possui o mesmo embedding independentemente de sua posição, portanto a informação posicional precisa ser adicionada para que o modelo consiga distinguir a ordem dos elementos dentro do contexto.

Ao final da Sprint, foi possível construir uma sequência completa de preparação:

**Texto → Tokens → Token IDs → Entrada/Alvo → Batches → Token Embeddings
→ Positional Embeddings → Input Embeddings**

Dessa forma, os resultados obtidos confirmam, de maneira prática, os conceitos apresentados no Capítulo 2 do livro *Build a Large Language Model (From Scratch)*.

O principal resultado desta Sprint é a obtenção dos *input embeddings*, que combinam as informações dos tokens com suas posições dentro do contexto. Essa representação será utilizada como entrada para as próximas etapas da arquitetura Transformer, especialmente para o mecanismo de self-attention.

Assim, a Sprint 2 estabelece a base necessária para a próxima etapa do projeto, na qual será estudado como o mecanismo de atenção utiliza essas representações para identificar e processar as relações entre os tokens de uma sequência.

4 Referências Bibliográficas

AN, Shengnan et al. *Make your llm fully utilize the context. Advances in Neural Information Processing Systems*, v. 37, p. 62160-62188, 2024.

RASCHKA, Sebastian. *Build a Large Language Model (From Scratch)*. Shelter Island: Manning Publications, 2024.

ZHAO, Wayne Xin et al. *A survey of large language models. Frontiers of Computer Science*, v. 20, n. 12, p. 2012627, 2026.